

NETWORK TELEMETRY MONITORING SYSTEM

Project Implementation Report

1. Project Overview

This project implements a network telemetry monitoring framework for collecting, transporting, storing, visualizing, and monitoring network performance metrics. SNMP-enabled Docker containers are used to simulate network devices. A Python collector periodically retrieves interface counters and interface speed using SNMP OIDs, measures latency using ICMP ping, calculates bandwidth and interface utilization, and sends the resulting telemetry to a central backend using gRPC. The backend stores telemetry in PostgreSQL. FastAPI provides REST endpoints for historical data access and CSV export, while Grafana provides visualization and threshold-based alerting.

1.1 Objectives

- Collect bandwidth-related and latency-related network telemetry at regular intervals.
- Use standard SNMP objects/OIDs for device-side metric collection.
- Process cumulative interface counters into bandwidth and utilization metrics.
- Transport telemetry from collectors to a central backend using gRPC and Protocol Buffers.
- Persist telemetry for historical analysis and reporting.
- Expose historical telemetry through REST APIs.
- Provide Grafana dashboards and threshold-based alerts.
- Support multiple monitored devices using reusable collector logic.

1.2 Technologies

Item	Details
Python	Collector, metric processing, gRPC backend and FastAPI API
SNMP / PySNMP	Network-device metric collection
gRPC / Protocol Buffers	Collector-to-backend telemetry transport
PostgreSQL	Historical telemetry storage
FastAPI / Uvicorn	REST API and interactive API documentation
Grafana	Dashboards and threshold-based alerting
Docker	Isolated simulated devices and application services
ICMP Ping	Latency measurement

2. System Architecture and Mechanism

The implementation follows a layered telemetry pipeline. Network-device information is exposed through SNMP. The collector gathers raw values and converts them into meaningful metrics. gRPC transports the telemetry to a central backend, which persists it in PostgreSQL. The stored data is consumed through FastAPI and Grafana.

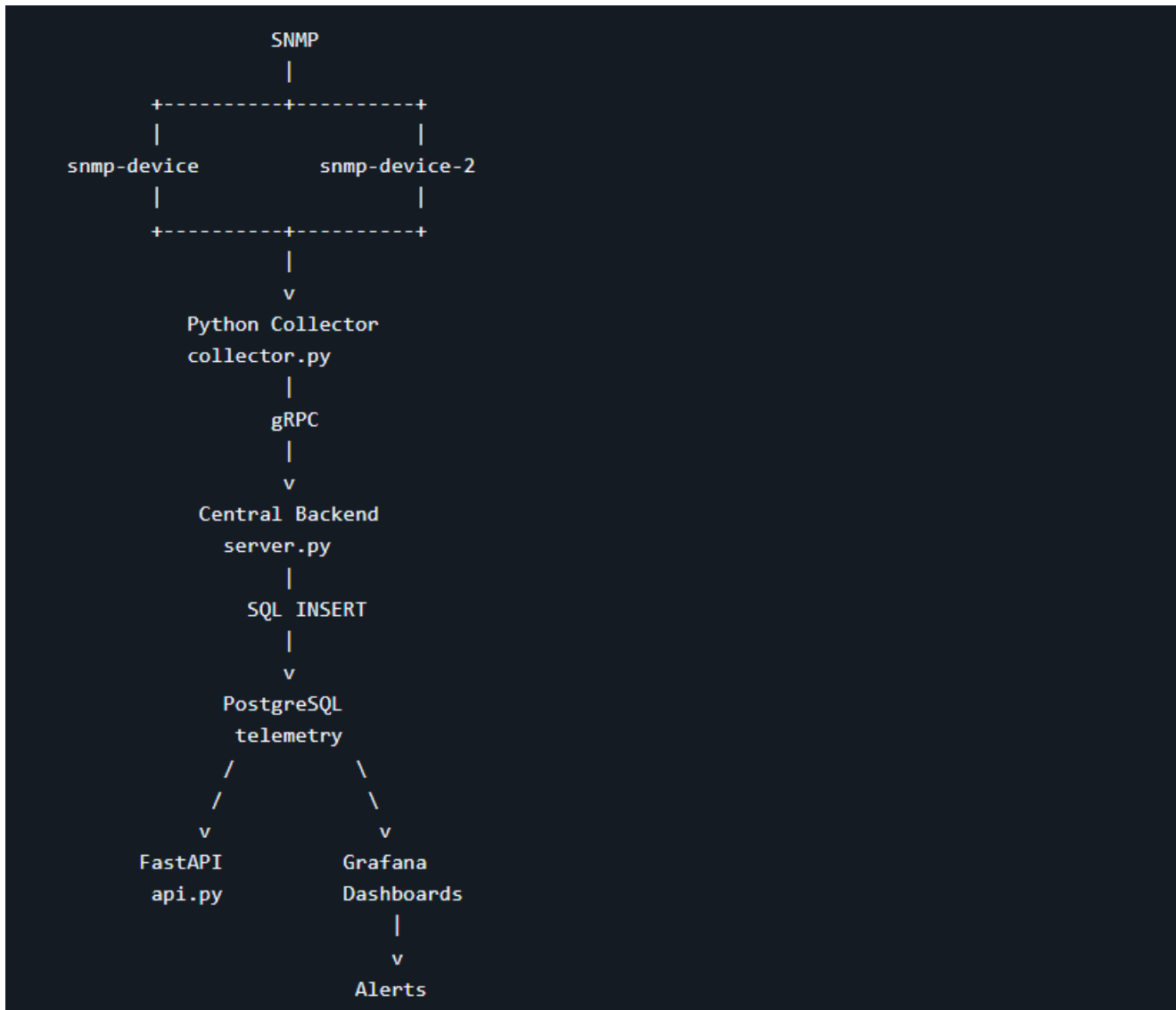


Figure 1. Overall network telemetry architecture.

2.1 SNMP Concepts Used

SNMP (Simple Network Management Protocol) is used as the device-facing collection protocol. The monitored device runs an SNMP agent, while the Python collector acts as the manager/client.

- MIB: a structured definition of manageable objects exposed by an SNMP agent.
- OID: the hierarchical identifier used to request a specific SNMP object.
- Community string: the SNMPv2c read credential used by the lab simulation.
- Interface counters: cumulative byte counters used to derive bandwidth over time.
- High-capacity counters are used for incoming and outgoing octets.

2.2 SNMP Configuration

The simulated SNMP agent is configured using the following file:

```
snmp/snmpd.conf/snmpd.conf

rocommunity public

sysLocation "Network Lab"
sysContact "student@localhost"

view all included .1
```

The configuration permits read-only access using the community string public and exposes the relevant SNMP object tree for the lab devices.

2.3 SNMP OIDs Used by the Collector

Item	Details
ifHCInOctets – interface 2 (eth0 is the interface)	1.3.6.1.2.1.31.1.1.1.6.2 - cumulative incoming octets
ifHCOutOctets – interface 2	1.3.6.1.2.1.31.1.1.1.10.2 - cumulative outgoing octets
ifHighSpeed – interface 2	1.3.6.1.2.1.31.1.1.1.15.2 - interface speed in Mbps

2.4 Simulated SNMP Devices Using Docker

Because the project is intended to monitor routers and switches but physical devices are not required for the laboratory implementation, two Docker containers are used as independent SNMP-enabled simulated devices.

```
docker network create network-lab

docker run -d \
  --name snmp-device \
  --network network-lab \
  -p 161:161/udp \
  -v "$(pwd)/snmp/snmpd.conf/snmpd.conf:/etc/snmp/snmpd.conf:ro" \
  polinux/snmpd

docker run -d \
  --name snmp-device-2 \
  --network network-lab \
  -v "$(pwd)/snmp/snmpd.conf/snmpd.conf:/etc/snmp/snmpd.conf:ro" \
  polinux/snmpd
```

The network-lab bridge network allows the collector to reach both devices using their container names. Only the first SNMP device publishes UDP port 161 to the host because both devices use the same internal port and the second device is accessed only through the Docker network.

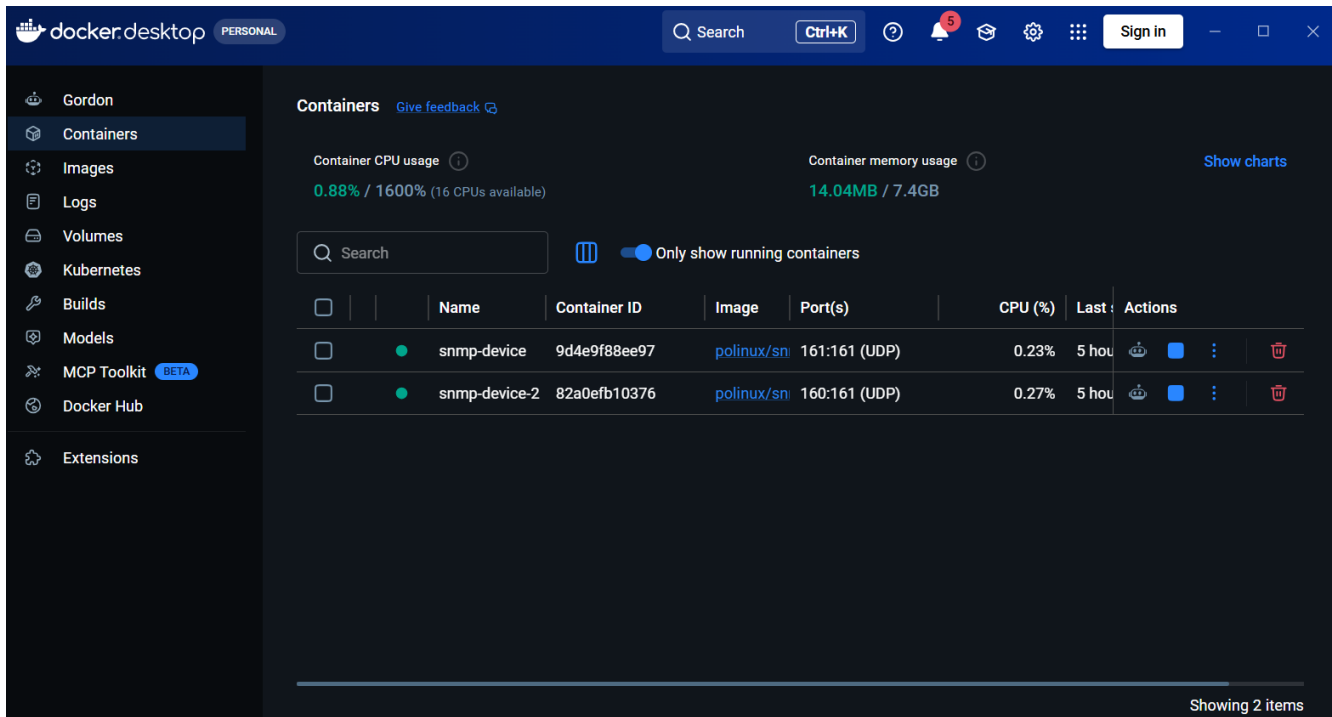


Figure 3. Running SNMP device containers.

3. Telemetry Collection and Metric Processing

The collector is implemented in `collector/collector.py`. It maintains a configurable list of monitored devices and reuses the same collection functions for every device.

```
devices = [
    {"name": "snmp-device", "host": "snmp-device", "port": 161},
    {"name": "snmp-device-2", "host": "snmp-device-2", "port": 161}
]
```

3.1 Periodic Collection

The collector processes each device in a loop and waits five seconds between collection cycles. This provides regular sampling without creating a separate collector implementation for each device.

```
for device in devices:
    ...
    await asyncio.sleep(5)
```

3.2 Bandwidth Calculation

SNMP octet counters are cumulative. Therefore, the collector does not interpret the current counter value as the traffic generated during the current interval. Instead, it subtracts the previous counter from the current counter and divides the resulting number of bits by elapsed time.

```
bandwidth_bps =
    ((current_counter - previous_counter) * 8) / elapsed_time
```

The collector stores previous readings independently for each device so that traffic from one device is never compared with counters from another device.

3.3 Interface Utilization

```
utilization_percent =  
    (bandwidth_bps / interface_capacity_bps) * 100
```

The interface speed is obtained from the ifHighSpeed OID and converted from Mbps to bits per second.

3.4 Latency Measurement

Latency is measured using one ICMP ping to the monitored device during each collection cycle. The collector extracts the response time and stores it as latency_ms.

3.5 Counter Reset/Wrap Protection

The collector checks whether a new SNMP counter value is smaller than the previous value. A decrease indicates a reset or wrap event, so the current reading is stored as the new baseline instead of calculating a false negative bandwidth value.

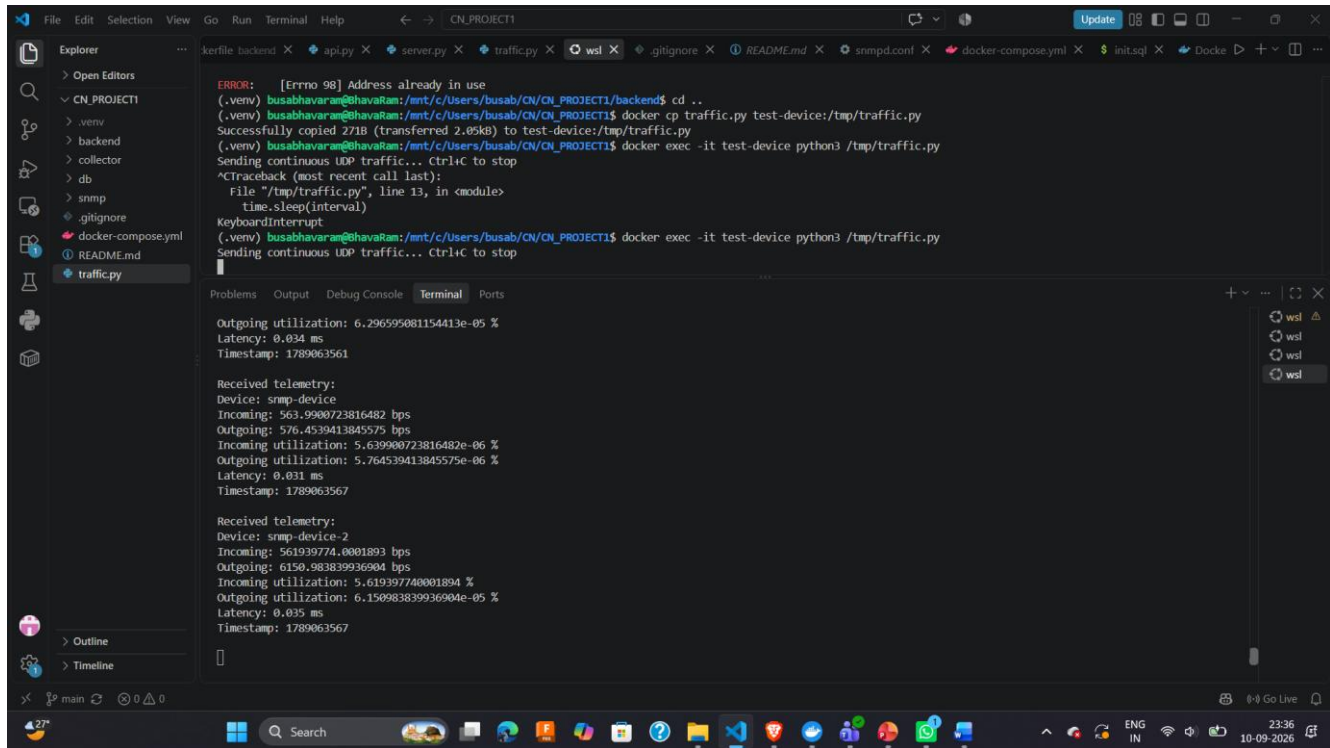


Figure 4. Collector processing and telemetry output.

4. Telemetry Transport Using gRPC

gRPC provides the service-to-service transport between the collector and the backend. The message structure and RPC contract are defined in `backend/proto/telemetry.proto`.

```
service TelemetryService {  
    rpc SendMetrics (TelemetryData) returns (TelemetryAck);  
}
```

```

message TelemetryData {
    string device_name = 1;
    double incoming_bps = 2;
    double outgoing_bps = 3;
    double incoming_utilization = 4;
    double outgoing_utilization = 5;
    double latency_ms = 6;
    int64 timestamp = 7;
}

```

The collector creates a gRPC channel to `backend:50051`, constructs a TelemetryData message, and calls SendMetrics(). The backend receives the request and returns a TelemetryAck response.

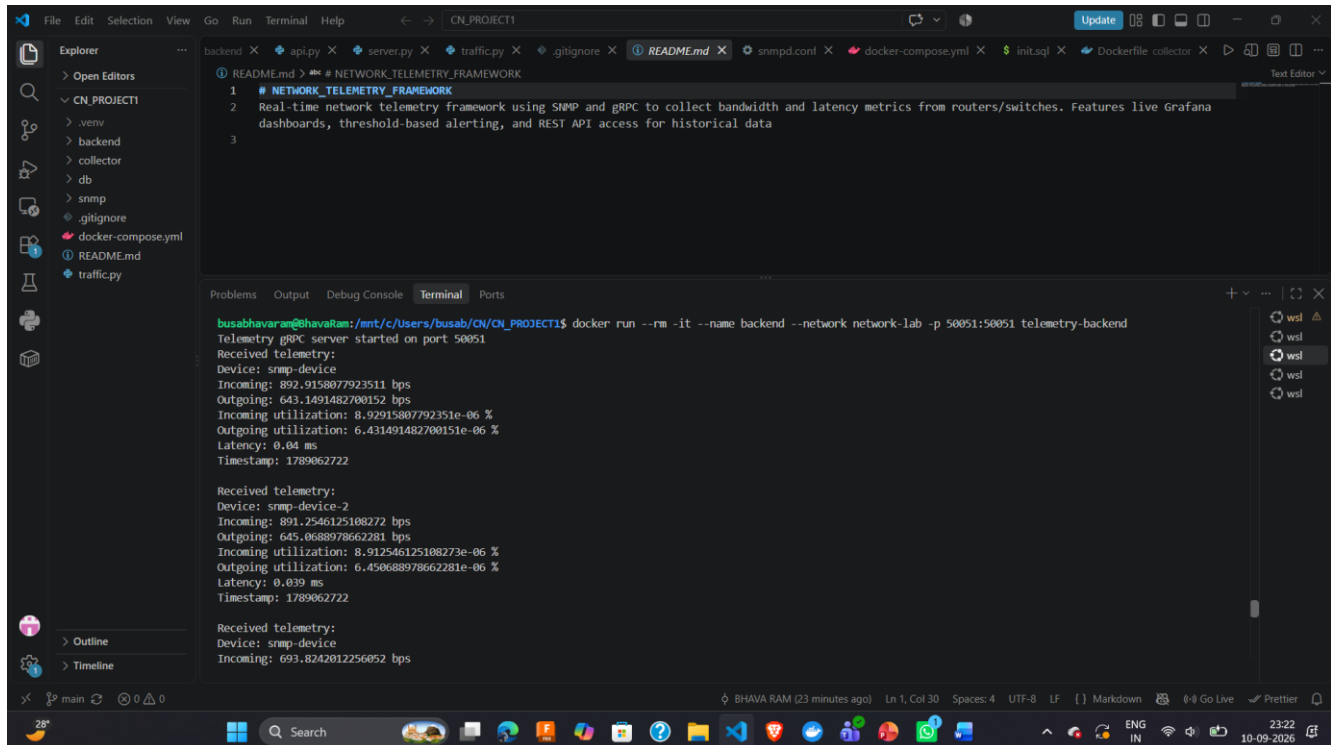


Figure 5. gRPC telemetry received by the central backend.

5. Historical Storage Using PostgreSQL

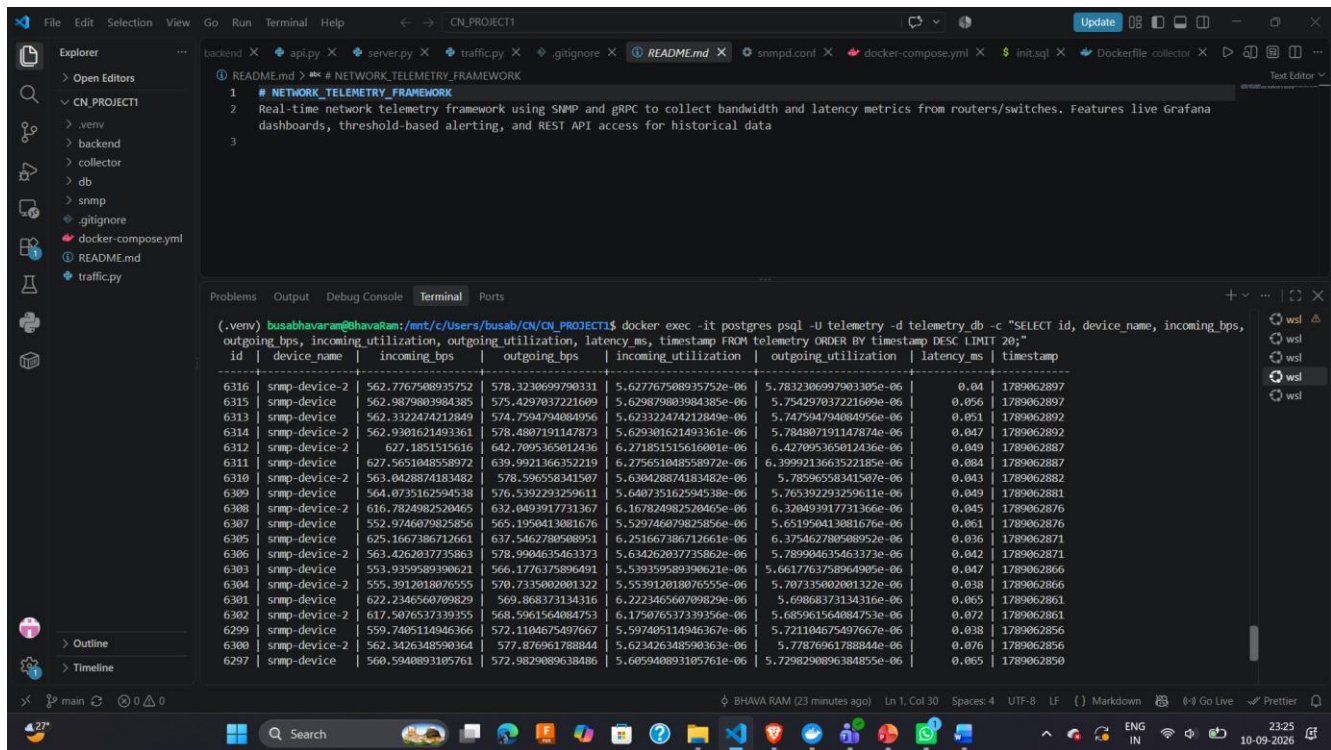
PostgreSQL provides persistent storage for the telemetry generated by the collector. The database schema is defined in `db/init.sql`.

```

CREATE TABLE IF NOT EXISTS telemetry (
    id BIGSERIAL PRIMARY KEY,
    device_name TEXT NOT NULL,
    incoming_bps DOUBLE PRECISION NOT NULL,
    outgoing_bps DOUBLE PRECISION NOT NULL,
    incoming_utilization DOUBLE PRECISION NOT NULL,
    outgoing_utilization DOUBLE PRECISION NOT NULL,
    latency_ms DOUBLE PRECISION NOT NULL,
    timestamp BIGINT NOT NULL
);

```

The backend performs an INSERT for every telemetry reading and commits the transaction. A composite index on device_name and timestamp supports device-specific time-series queries.



The screenshot shows a VS Code editor with a terminal window open. The terminal displays a SQL query and its results. The query is: `SELECT id, device_name, incoming_bps, outgoing_bps, incoming_utilization, outgoing_utilization, latency_ms, timestamp FROM telemetry ORDER BY timestamp DESC LIMIT 20;` The results are displayed in a table with 8 columns: id, device_name, incoming_bps, outgoing_bps, incoming_utilization, outgoing_utilization, latency_ms, and timestamp. The table contains 20 rows of data, showing telemetry readings for various devices over time.

id	device_name	incoming_bps	outgoing_bps	incoming_utilization	outgoing_utilization	latency_ms	timestamp
6316	snmp-device-2	562.7767508935752	578.3230699790331	5.627767508935752e-06	5.7832306997903305e-06	0.04	1789062897
6315	snmp-device	562.9879803984385	575.4297037221609	5.629879803984385e-06	5.754297037221609e-06	0.056	1789062897
6313	snmp-device	562.3322474212849	574.7594794084956	5.623322474212849e-06	5.747594794084956e-06	0.051	1789062892
6314	snmp-device-2	562.9301621493361	578.4807191147873	5.629301621493361e-06	5.784807191147874e-06	0.047	1789062892
6312	snmp-device-2	627.1851515616	642.7095365012436	6.271851515616001e-06	6.427095365012436e-06	0.049	1789062887
6311	snmp-device	627.5651048558972	639.9921366352219	6.275651048558972e-06	6.3999213663522185e-06	0.084	1789062887
6310	snmp-device-2	563.0428874183482	578.596558341507	5.630428874183482e-06	5.78596558341507e-06	0.043	1789062882
6309	snmp-device	564.0735162594538	576.5392293259611	5.640735162594538e-06	5.765392293259611e-06	0.049	1789062881
6308	snmp-device-2	616.7824982520465	632.0493917731367	6.167824982520465e-06	6.320493917731366e-06	0.045	1789062876
6307	snmp-device	552.9746079825856	565.1950413081676	5.529746079825856e-06	5.651950413081676e-06	0.061	1789062876
6305	snmp-device	625.1667386712661	637.5462780508951	6.251667386712661e-06	6.375462780508952e-06	0.036	1789062871
6306	snmp-device-2	563.4262037735863	578.9904635463373	5.634262037735862e-06	5.789904635463373e-06	0.042	1789062871
6303	snmp-device	553.9359589390621	566.1770375896401	5.539359589390621e-06	5.6617703758964005e-06	0.047	1789062866
6304	snmp-device-2	555.3912018076555	570.7335802001322	5.553912018076555e-06	5.707335802001322e-06	0.038	1789062866
6301	snmp-device	622.2346560709829	569.868373134316	6.22246560709829e-06	5.68868373134316e-06	0.065	1789062861
6302	snmp-device-2	617.5076537339355	568.5961564084753	6.175076537339355e-06	5.685961564084753e-06	0.072	1789062861
6299	snmp-device	559.7405114946366	572.1104675497667	5.597405114946367e-06	5.721104675497667e-06	0.038	1789062856
6300	snmp-device-2	562.3426348590364	577.876961788844	5.623426348590363e-06	5.77876961788844e-06	0.076	1789062856
6297	snmp-device	560.5940893105761	572.9829089638486	5.605940893105761e-06	5.7298290896384855e-06	0.065	1789062850

Figure 6. Telemetry persisted in PostgreSQL.

6. REST API Using FastAPI

FastAPI is used as the REST interface to the stored telemetry. It is separate from the gRPC server: server.py receives telemetry, while api.py provides clients with access to historical data.

6.1 Retrieve Historical Telemetry

GET `http://localhost:8000/telemetry`

The endpoint queries the telemetry table and returns the stored records as JSON.

6.2 Export Historical Telemetry

GET `http://localhost:8000/telemetry/export`

The endpoint reads the telemetry records, writes them to an in-memory CSV stream, and returns the CSV response.

6.3 Interactive API Documentation

`http://localhost:8000/docs`

FastAPI automatically provides interactive API documentation through Swagger UI.

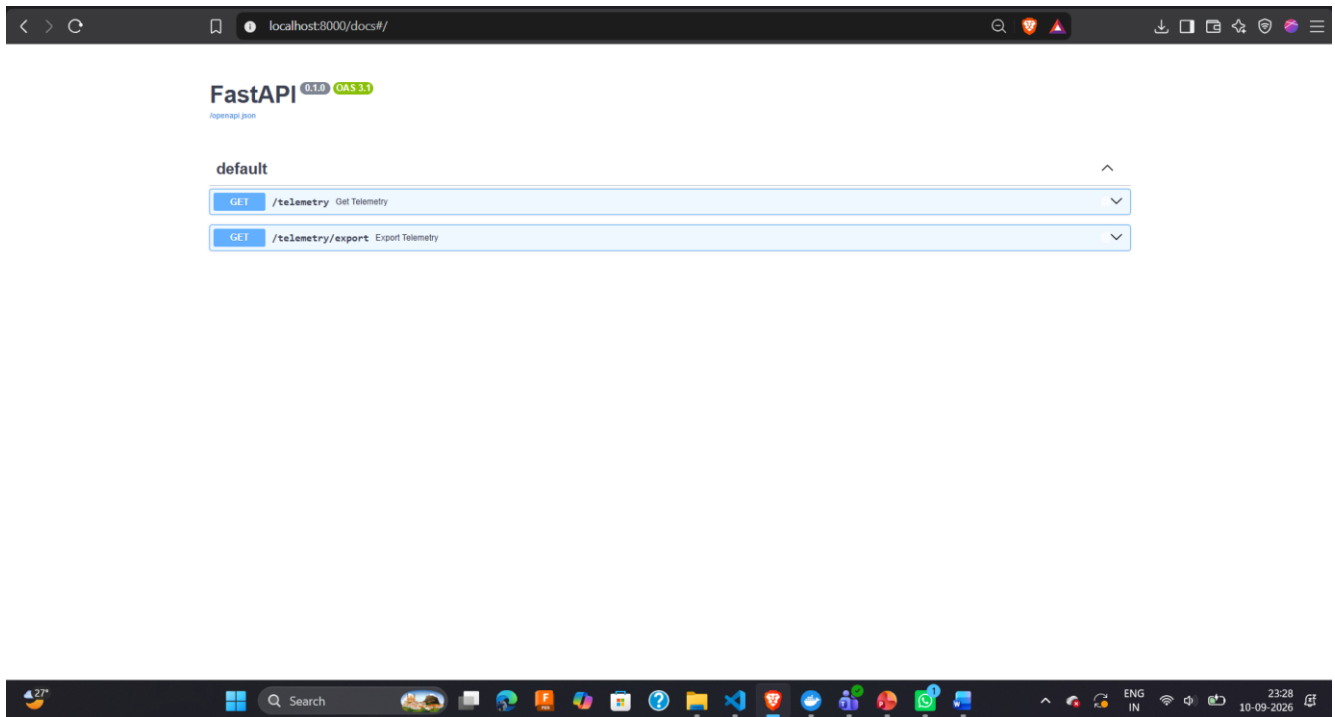


Figure 7. FastAPI interactive REST API documentation.

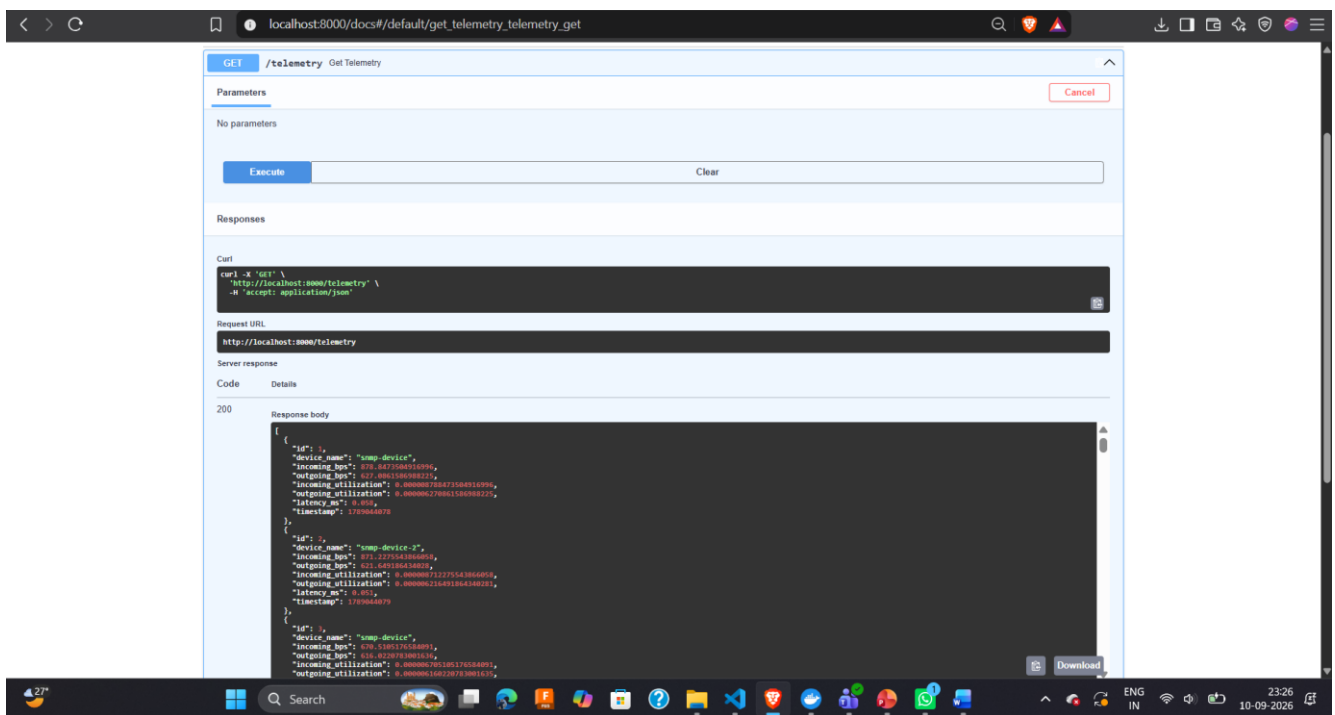


Figure 8. Historical telemetry returned through the REST API.

7. Visualization and Alerting Using Grafana

Grafana is connected to PostgreSQL and is used to visualize the stored telemetry as time-series panels. The dashboard can be filtered by device using a dashboard variable populated from the telemetry table.

7.1 Dashboard Variable

```
SELECT DISTINCT device_name
FROM telemetry
ORDER BY device_name;
```

This query populates a device selector that allows users to switch between snmp-device and snmp-device-2 without creating separate dashboards for every device.

7.2 Incoming Utilization Panel

```
SELECT
    to_timestamp(timestamp) AS "time",
    device_name,
    incoming_utilization
FROM telemetry
WHERE timestamp >= $__unixEpochFrom()
    AND timestamp <= $__unixEpochTo()
ORDER BY timestamp;
```

7.3 Outgoing Utilization Panel

```
SELECT
    to_timestamp(timestamp) AS "time",
    device_name,
    outgoing_utilization
FROM telemetry
WHERE timestamp >= $__unixEpochFrom()
    AND timestamp <= $__unixEpochTo()
ORDER BY timestamp;
```

7.4 Latency Panel

```
SELECT
    to_timestamp(timestamp) AS "time",
    device_name,
    latency_ms
FROM telemetry
WHERE timestamp >= $__unixEpochFrom()
    AND timestamp <= $__unixEpochTo()
ORDER BY timestamp;
```

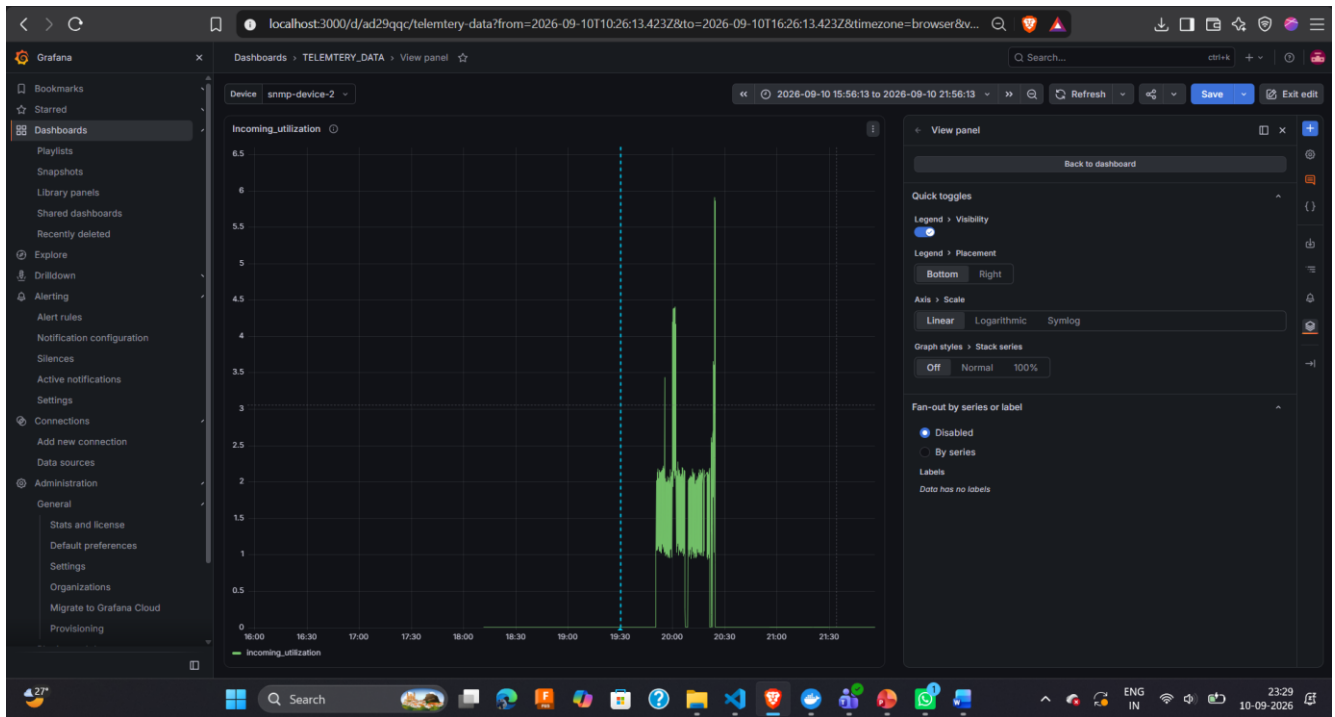


Figure 9. Incoming utilization visualization.

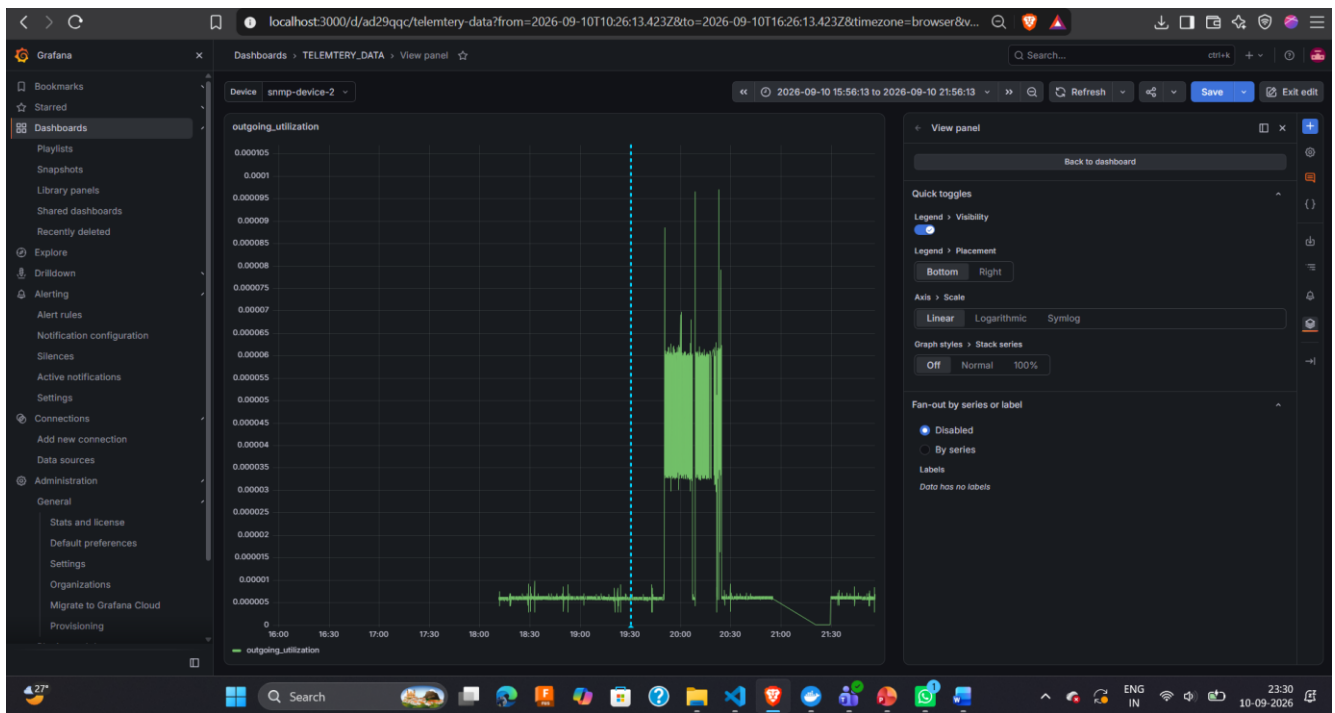


Figure 10. Outgoing utilization visualization.

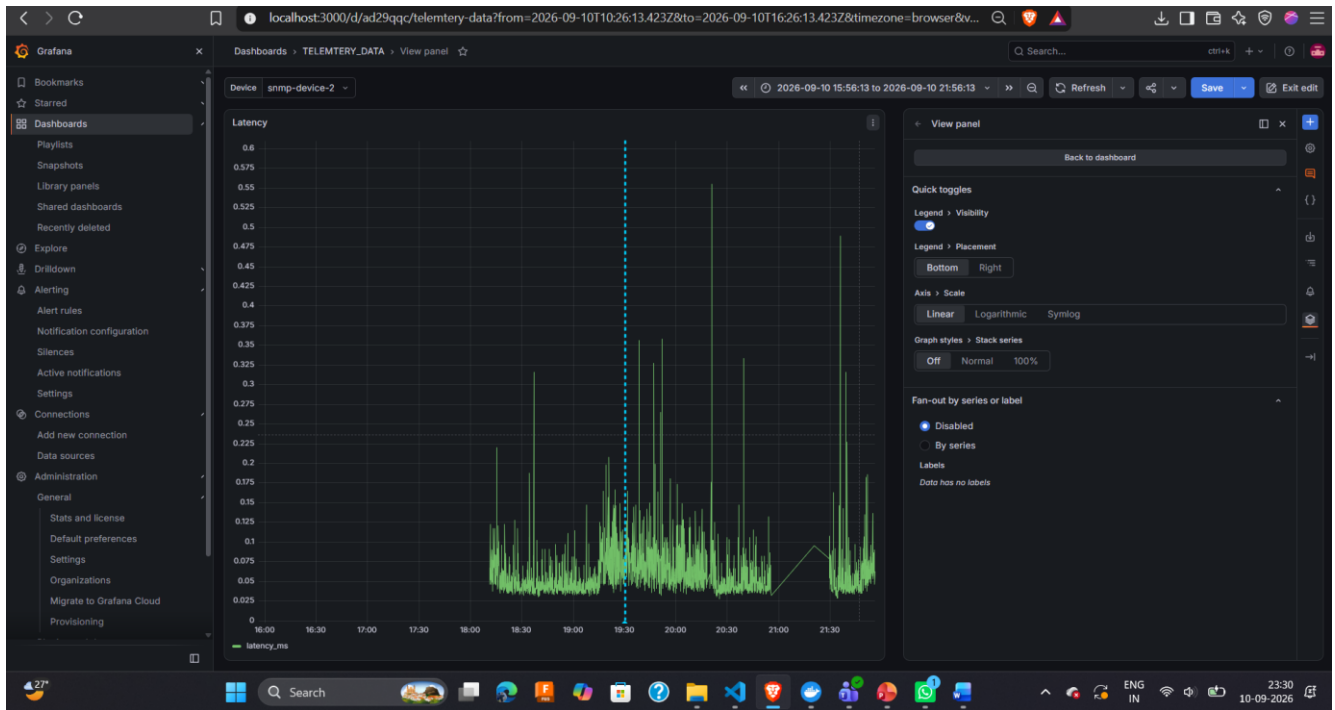


Figure 11. Latency visualization.

7.5 Threshold-Based Alerting

Grafana alert rules are configured for the utilization and latency panels. The alert condition uses the most recent value (Last) and a threshold. For demonstration and testing, low thresholds can be used to force the rule into the Firing state; meaningful operational thresholds can then be used for normal monitoring.

Item	Details
Reducer	Last
Pending period	0 seconds for immediate triggering
Keep firing for	5 seconds in the tested configuration
Evaluation interval	10 seconds in the tested configuration
Example operational thresholds	Utilization > 80%; latency > 100 ms

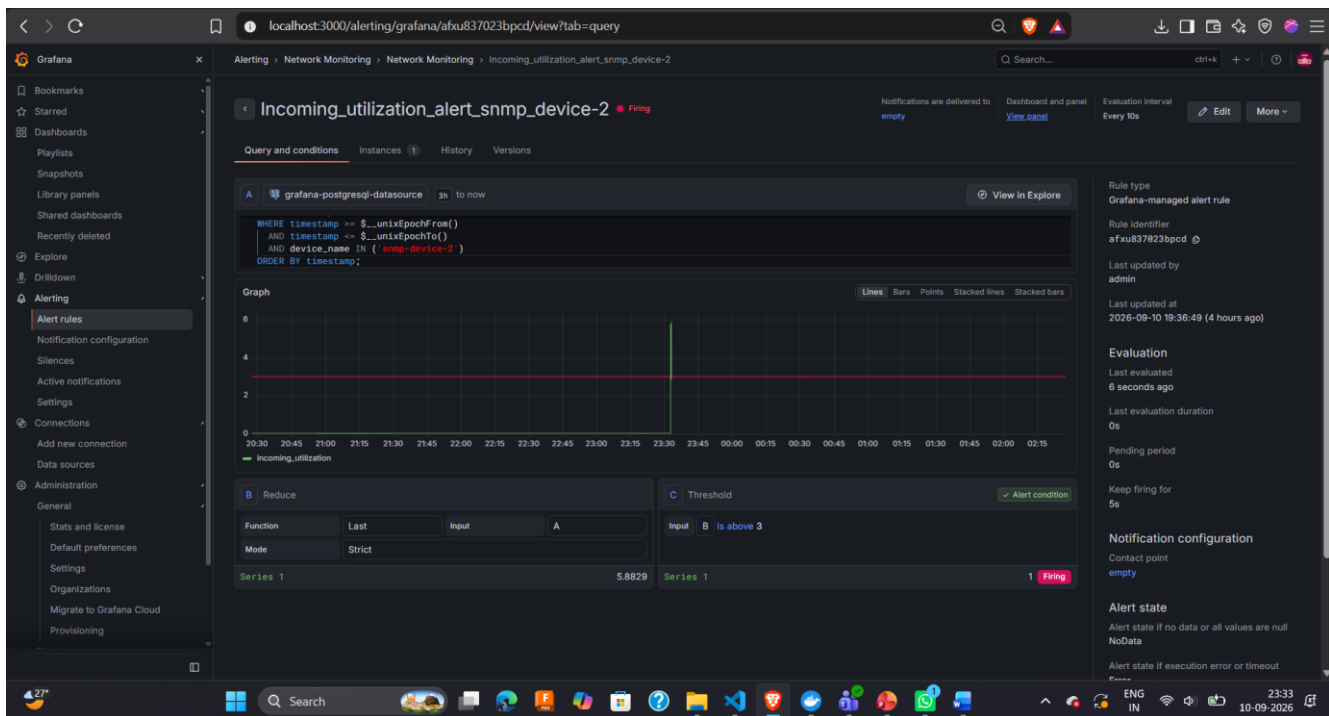


Figure 12. Grafana threshold-based alert firing.

8. Traffic Generation and Functional Testing

`traffic.py` is retained as a testing utility. It generates UDP traffic toward the simulated device so that interface counters change and the collector can observe non-zero bandwidth and utilization.

The intended verification chain is:

```

traffic.py
↓
Network traffic reaches the simulated device interface
↓
SNMP counters increase
↓
collector.py detects counter differences
↓
bandwidth / utilization increase
↓
backend stores telemetry
↓
Grafana reflects the change
↓
threshold alert can fire

```

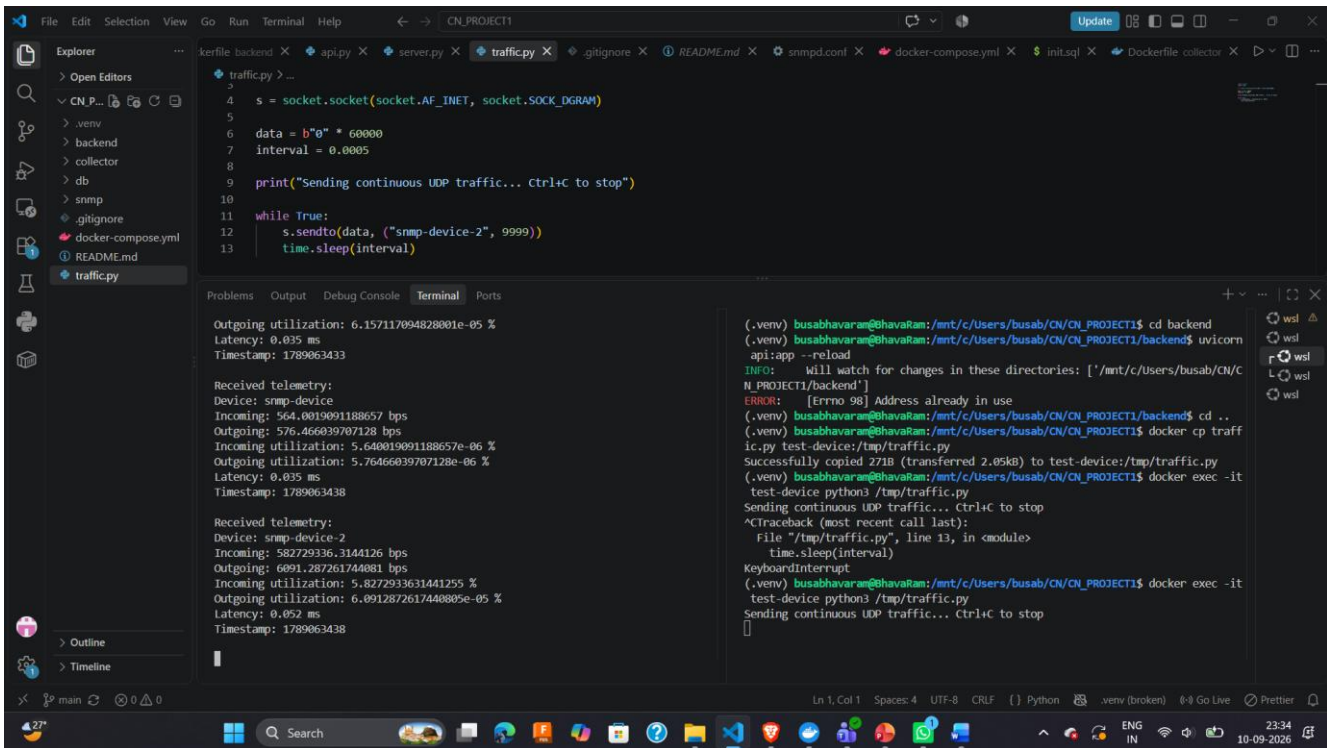


Figure 13. Traffic generation used to validate telemetry changes.

9. End-to-End Execution Procedure

The working demonstration can be started manually without depending on Docker Compose. The following sequence creates an isolated Docker network, starts the simulated devices and database, builds the application images, and runs the collector/backend/API/Grafana components.

Create the Docker network

```
docker network create network-lab
```

Creates an isolated bridge network used for inter-container communication.

Start SNMP device 1

```
docker run -d --name snmp-device --network network-lab -p 161:161/udp -v
"${pwd}/snmp/snmpd.conf:/etc/snmp/snmpd.conf:ro" polinux/snmpd
```

Starts the first simulated SNMP device.

Start SNMP device 2

```
docker run -d --name snmp-device-2 --network network-lab -v
"${pwd}/snmp/snmpd.conf:/etc/snmp/snmpd.conf:ro" polinux/snmpd
```

Starts the second simulated SNMP device on the internal network.

Start PostgreSQL

```
docker run -d --name postgres --network network-lab -e POSTGRES_USER=telemetry -e
POSTGRES_PASSWORD=telemetry123 -e POSTGRES_DB=telemetry_db -p 5432:5432 -v postgres-
```

```
data:/var/lib/postgresql/data -v "$(pwd)/db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro"
postgres:17
```

Starts PostgreSQL and mounts the database initialization script.

Build backend image

```
docker build -f backend/Dockerfile -t telemetry-backend .
```

Builds the image containing server.py and api.py.

Build collector image

```
docker build -f collector/Dockerfile -t network-collector .
```

Builds the image containing collector.py and its Python dependencies.

Start backend

```
docker run -d --name backend --network network-lab -p 50051:50051 telemetry-backend
```

Starts the gRPC backend on port 50051.

Start FastAPI

```
docker run -d --name api --network network-lab -p 8000:8000 telemetry-backend uvicorn api:app --host
0.0.0.0 --port 8000
```

Runs api.py through Uvicorn on port 8000.

Start collector

```
docker run -d --name collector --network network-lab network-collector
```

Starts the telemetry collection loop.

Start Grafana

```
docker run -d --name grafana --network network-lab -p 3000:3000 grafana/grafana
```

Starts Grafana on port 3000.

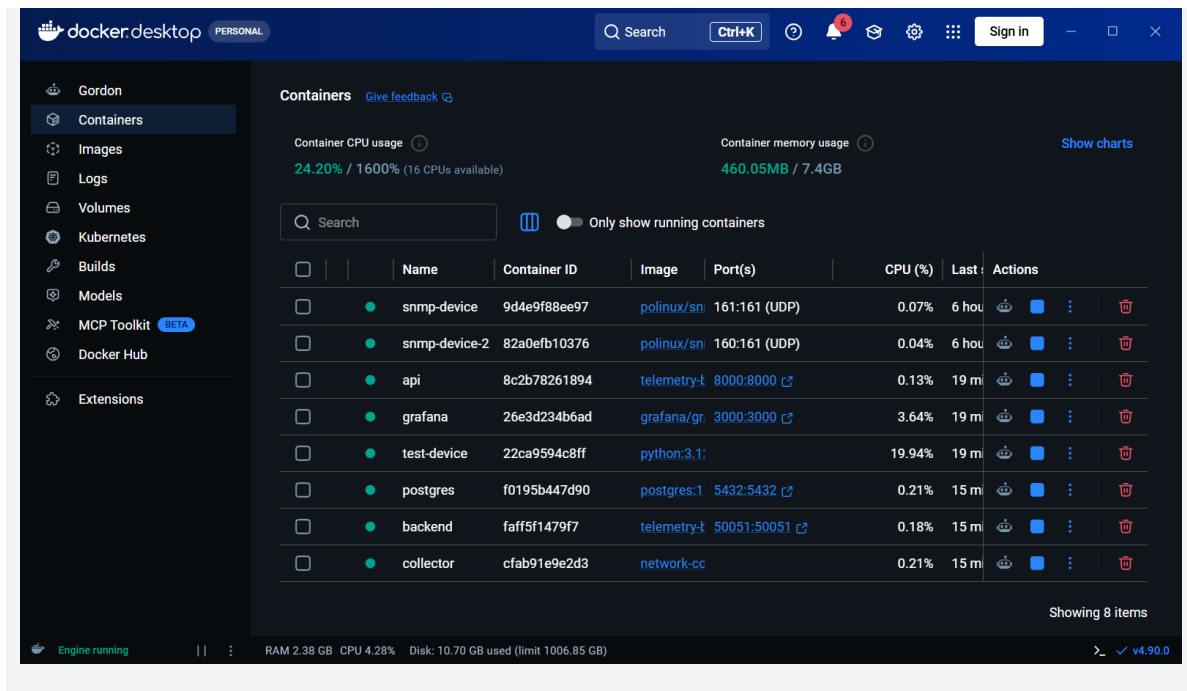


Figure 14. Complete running container set.

10. Evaluation and Results

The implementation is evaluated by following the telemetry from collection to visualization. Successful operation is indicated by the collector producing measurements, the backend acknowledging SendMetrics requests, PostgreSQL receiving new rows, the REST API returning the stored data, Grafana displaying time-series data, and the alert rule entering the Firing state when a threshold is crossed.

Item	Details
Metric collection	SNMP counters and interface speed retrieved for configured devices
Bandwidth	Computed from consecutive cumulative counter readings
Utilization	Computed from throughput relative to interface capacity
Latency	Measured with ICMP ping
Transport	gRPC SendMetrics RPC
Persistence	PostgreSQL telemetry table
Historical access	FastAPI /telemetry
Export	FastAPI /telemetry/export
Visualization	Grafana time-series panels
Alerting	Grafana threshold rules
Scalability	Reusable collector logic over a configurable device list

11. Issues Identified and Fixed During Development

Counter handling

Added protection against decreasing SNMP counters so resets/wraps do not create invalid negative bandwidth values.

Multiple devices

Changed previous-reading storage to be device-specific, allowing a single collector to monitor multiple devices.

gRPC integration

Introduced telemetry.proto and generated Python gRPC/protobuf modules for a defined collector-to-backend contract.

Backend persistence

Connected the gRPC backend to PostgreSQL so telemetry is retained instead of existing only in memory.

REST access

Added FastAPI endpoints for JSON retrieval and CSV export of historical telemetry.

Docker networking

Used a shared Docker network and container names so collector/backend/database components can communicate without hard-coded container IPs.

Grafana time filtering

Used Unix-epoch-aware Grafana time macros because the PostgreSQL timestamp field is stored as BIGINT epoch seconds.

12. Limitations and Future Extensions

- The current lab implementation uses SNMPv2c with a read-only community string; SNMPv3 can be introduced for authenticated/encrypted production communication.
- The collector currently targets a configured interface index (interface 2) for the demonstrated SNMP counters.
- The simulated Docker SNMP devices can be replaced by authorized physical or virtual SNMP-enabled routers/switches by supplying their network addresses and credentials.
- A more production-oriented implementation could use centralized configuration or service discovery instead of editing the device list manually.
- OpenTelemetry can be added later if broader observability integration is required.

13. Conclusion

The completed system demonstrates an end-to-end network telemetry pipeline. SNMP is used to obtain device and interface metrics, the Python collector transforms raw counters into bandwidth and utilization values and measures latency, gRPC transfers the processed telemetry to a central backend, PostgreSQL

preserves historical measurements, FastAPI exposes the stored data through REST, and Grafana provides visualization and threshold-based alerting. The use of a configurable device list and reusable service components also provides a foundation for extending the system to additional devices and metrics.